



งานปฏิบัติการกลุ่ม StackQueue

ผู้จัดทำ

นาย ณัฐนันท์ เอ่งฉ้วน 68122250001

นาย พุฒิชัย โชติพัทธ์จำ 68122250094

นาย ธนกร ชุมเดชา 68122250029

นาย ตะวัน นามลนวน 67122250094

เสนอ

ผศ.ดร. กัลยณัฐ กุหลาบเพชรทอง

รายงานนี้เป็นส่วนหนึ่งของวิชา CSD2103

การออกแบบและวิเคราะห์ขั้นตอนวิธี

สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏสวนสุนันทา

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ในการปฏิบัติงานด้านเหตุฉุกเฉินหรือ Emergency Workflow จะมีลำดับขั้นตอนการทำงานที่ชัดเจน เช่น การรับแจ้งเหตุ การมอบหมายทีม การส่งยานพาหนะ การเดินทางถึงสถานที่เกิดเหตุ และการปิดเคส แต่ละขั้นตอนมีความสัมพันธ์กับขั้นตอนก่อนหน้านี้ จึงไม่ควรเปลี่ยนไปยังขั้นตอนถัดไปอย่างอิสระ

โครงงานนี้นำโครงสร้างข้อมูล Stack มาใช้จัดการ Event และรองรับ Undo/Redo ตามหลัก LIFO (Last In, First Out) โดยเปรียบเทียบ 2 วิธี ได้แก่ Algorithm A: Event Stack และ Algorithm B: Event Stack + State Machine ทั้งสองวิธีใช้ Stack สำหรับจัดเก็บ Event และ Redo Stack แต่ต่างกันที่การจัดการสถานะปัจจุบัน โดย Algorithm A อ่านสถานะจากข้อมูลบน Stack ส่วน Algorithm B เก็บสถานะไว้ในตัวแปร currentState โดยตรง โปรแกรมพัฒนาด้วยภาษา Java และใช้ ArrayDeque เป็นโครงสร้างพื้นฐานของ Stack.

1.2 Case Story

Emergency Workflow: สมมติว่าศูนย์รับแจ้งเหตุฉุกเฉินได้รับการแจ้งเหตุ ระบบต้องทำงานตามลำดับดังนี้

- 1) CALL_RECEIVED: NEW → RECEIVED
- 2) TEAM_ASSIGNED: RECEIVED → ASSIGNED
- 3) VEHICLE_DISPATCHED: ASSIGNED → DISPATCHED
- 4) ARRIVED_AT_SCENE: DISPATCHED → ON_SCENE
- 5) CASE_CLOSED: ON_SCENE → CLOSED

ดังนั้น Workflow ที่ถูกต้องคือ NEW → RECEIVED → ASSIGNED → DISPATCHED → ON_SCENE → CLOSED ระบบไม่อนุญาตให้ผู้ใช้งานข้ามขั้นตอนหรือดำเนินการผิดพลาด โดยกำหนดการเปลี่ยนสถานะไว้ใน Transition Table.

1.3 วัตถุประสงค์

1. เพื่อศึกษาและประยุกต์ใช้โครงสร้างข้อมูล Stack กับปัญหา Emergency Workflow
2. เพื่อออกแบบอัลกอริทึมสำหรับจัดการ Event และ Workflow
3. เพื่อเปรียบเทียบ Algorithm A และ Algorithm B

4. เพื่อศึกษาการทำงานของ Undo และ Redo ด้วย Stack
5. เพื่อศึกษาการควบคุมลำดับขั้นตอนด้วย State Machine
6. เพื่อวิเคราะห์ Time Complexity และ Space Complexity
7. เพื่อทดลองและเปรียบเทียบประสิทธิภาพของอัลกอริทึม
8. เพื่อพัฒนาโปรแกรมภาษา Java ที่สามารถจำลองระบบ Emergency Workflow ได้

1.4 ขอบเขตงาน

1. รองรับ Workflow จำนวน 5 ขั้นตอน และ 6 สถานะ ได้แก่ NEW, RECEIVED, ASSIGNED, DISPATCHED, ON_SCENE และ CLOSED
2. รองรับ Event 5 ประเภท ได้แก่ CALL_RECEIVED, TEAM_ASSIGNED, VEHICLE_DISPATCHED, ARRIVED_AT_SCENE และ CASE_CLOSED
3. รองรับการเพิ่ม Action, Undo, Redo, Reset และการแสดงข้อมูล Stack
4. สามารถเปรียบเทียบ Algorithm A และ Algorithm B และทดสอบประสิทธิภาพ
5. โปรแกรมพัฒนาด้วยภาษา Java รองรับ Java 8 ขึ้นไป

บทที่ 2 การวิเคราะห์ปัญหา

2.1 Input

ข้อมูลนำเข้าประกอบด้วย Action และคำสั่งจากผู้ใช้ เช่น เลือก Algorithm, เพิ่ม Action, Undo, Redo, ดู Stack/State, Reset, Test Cases, เปรียบเทียบ Algorithm A/B, Performance Test และ Exit

2.2 Output

ผลลัพธ์ประกอบด้วยสถานะปัจจุบันของ Workflow, รายการ Event ใน Stack, รายการ Event ที่สามารถ Redo ได้, ผลการเพิ่ม Action, Undo, Redo, การตรวจสอบ Action ผิดลำดับ, การ Reset, ผลการทดสอบ และผลการเปรียบเทียบประสิทธิภาพ

2.3 Constraints

1. Action ต้องเป็น Action ที่กำหนดไว้ในระบบ
2. Action ต้องดำเนินการตามลำดับ Workflow
3. ไม่สามารถเพิ่ม Action ที่ผิดลำดับได้

4. ไม่สามารถ Undo เมื่อ Stack ว่าง
5. ไม่สามารถ Redo เมื่อ Redo Stack ว่าง
6. เมื่อเพิ่ม Action ใหม่หลัง Undo แล้ว ข้อมูล Redo ต้องถูกล้าง
7. เมื่ออยู่ในสถานะ CLOSED จะไม่สามารถเพิ่ม Action ต่อได้
8. ต้องรักษาความถูกต้องของสถานะระหว่าง Undo และ Redo

2.4 Assumptions

กำหนดให้มี Emergency Case หนึ่งรายการต่อการทำงานหนึ่งชุด Event ถูกเพิ่มตามลำดับเวลา แต่ละ Event บันทึก previousState และ nextState การ Undo นำ Event ล่าสุดออกจาก Stack การ Redo นำ Event ที่ Undo กลับเข้าสู่ Stack และ State Machine เป็นตัวกำหนดว่า Action ใดสามารถเกิดขึ้นใน State ใด ผู้ใช้ป้อนคำสั่งผ่าน Console และการวัดประสิทธิภาพใช้ System.nanoTime()

2.5 Edge Cases

1. Stack ว่าง: Undo ต้องไม่ทำให้โปรแกรมเกิดข้อผิดพลาด
2. Redo Stack ว่าง: Redo ต้องคืนค่าไม่สำเร็จ
3. Action ผิดลำดับ: เช่น RECEIVED แล้วเลือก CASE_CLOSED ต้องถูกปฏิเสธ
4. Undo แล้วเพิ่ม Action ใหม่: ต้องล้างข้อมูล Redo เพราะเป็นเส้นทาง Workflow ใหม่
5. Redo แล้ว State ไม่ตรงกัน: ต้องตรวจสอบ State ก่อนนำ Event กลับเข้าสู่ Stack

บทที่ 3 การออกแบบอัลกอริทึม

3.1 Algorithm A: Event Stack

ใช้ Stack เป็นแหล่งข้อมูลหลักสำหรับจัดเก็บ Event โดย Event เก็บ Action, Previous State และ Next State สถานะปัจจุบันคำนวณจาก Event บนสุดของ Stack และถ้า Stack ว่างจะเป็น NEW

การเพิ่ม Action: ตรวจสอบ Action → อ่าน State จาก Stack → หา Next State จาก Transition Table → ถ้าไม่พบ Transition ให้ปฏิเสธ → สร้าง Event → Push ลง Stack → ล้าง Redo Stack

Undo: Pop Event ล่าสุด → ถ้าไม่มี Event ให้ล้มเหลว → Push Event ไป Redo Stack

Redo: Pop จาก Redo Stack → ตรวจสอบ State ปัจจุบันและ Transition → ถ้าตรงกัน Push กลับ Stack ถ้าไม่ตรงกันคืน Event ไป Redo Stack

3.2 Algorithm B: Event Stack + State Machine

ใช้ Stack เช่นเดียวกับ Algorithm A แต่เก็บสถานะปัจจุบันไว้ในตัวแปร currentState เริ่มต้นเป็น NEW เมื่อเพิ่ม Action สำเร็จ currentState จะเปลี่ยนเป็น next และเมื่อ Undo จะเปลี่ยนกลับเป็น previousState ทำให้เข้าถึงสถานะปัจจุบันได้โดยตรง

3.3 Pseudocode

Algorithm A

addAction(action)

- if action == null return false
- current = getCurrentState()
- next = table.nextState(current, action)
- if next == null return false
- push Event(action, current, next)
- clear Redo Stack
- return true

undo()

- pop Event จาก Stack
- ถ้าไม่มี Event return false
- push Event ไป Redo Stack
- return true

redo()

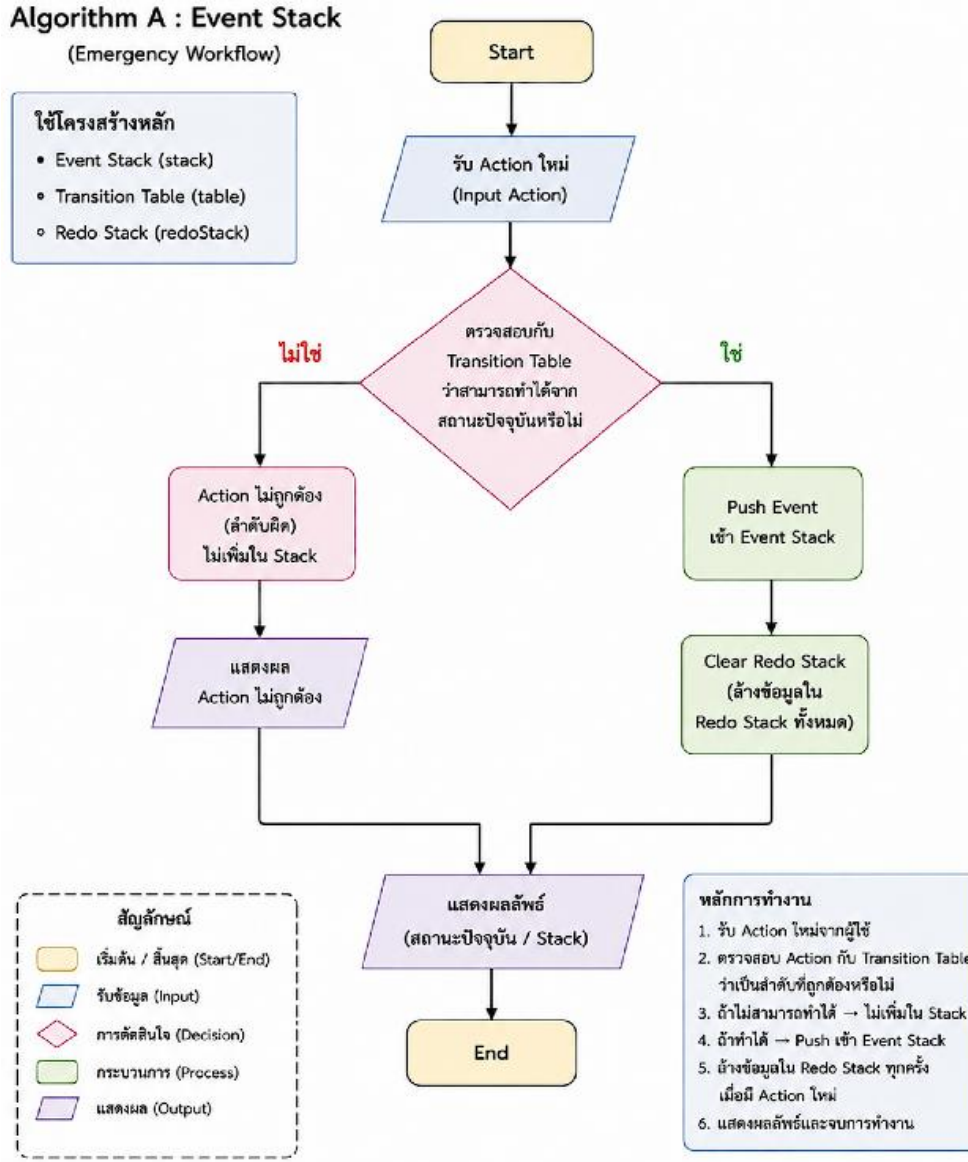
- pop Event จาก Redo Stack
- ถ้าไม่มี Event return false
- ตรวจสอบ nextState จาก Transition Table
- ถ้าไม่ตรงกับ Event เดิม ให้คืน Event ไป Redo Stack และ return false

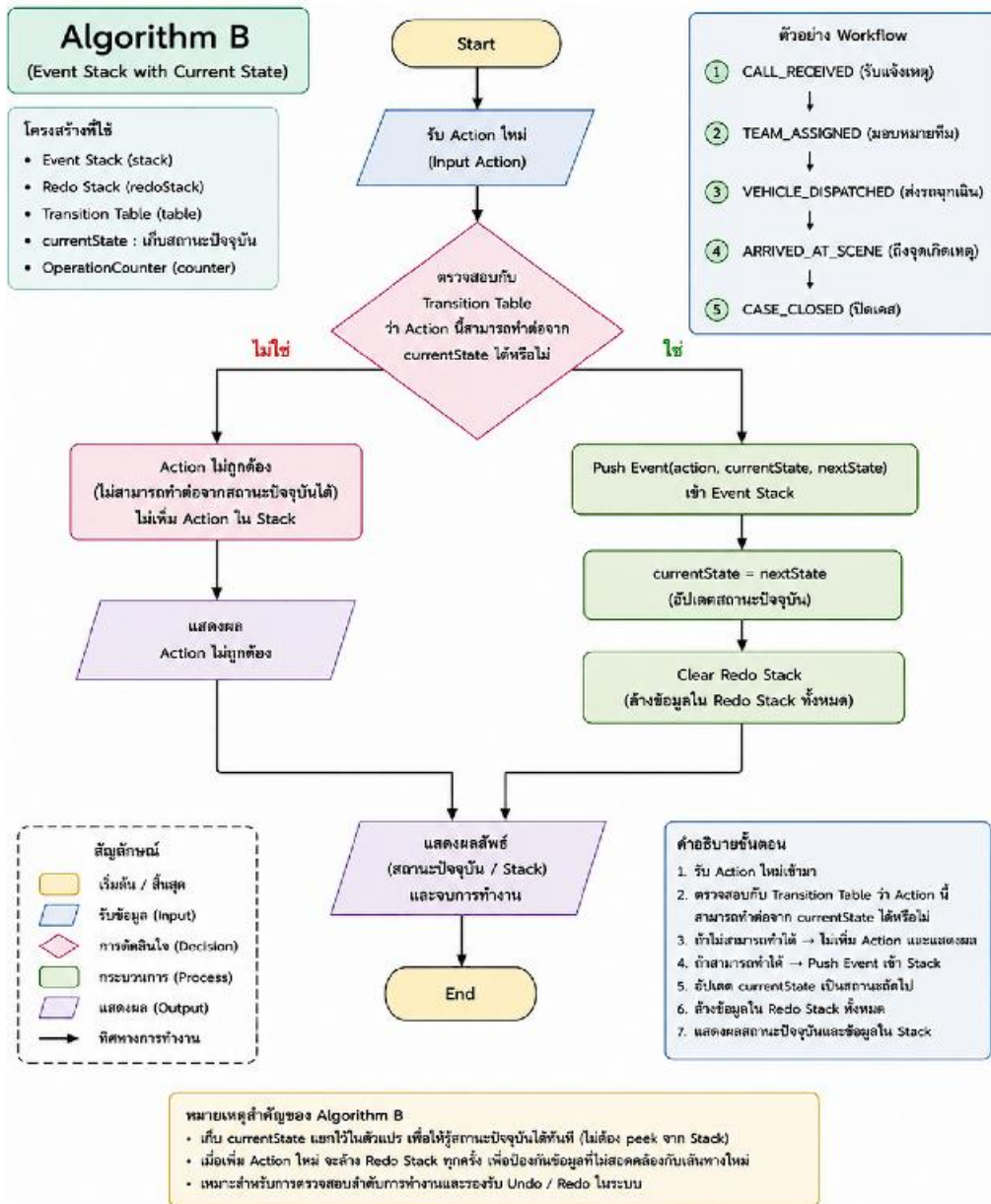
- push Event กลับ Stack
- return true

Algorithm B ใช้ขั้นตอนเดียวกัน แต่ใช้ currentState โดยตรง และอัปเดต currentState เมื่อ Add, Undo และ Redo สำเร็จ

3.4 Flowchart

Algorithm A : Event Stack
(Emergency Workflow)





3.5 Step-by-step และสถานะ Stack

การทำงานของโปรแกรมแบบ Step-by-Step

โปรแกรมนี้ใช้สำหรับจัดการลำดับเหตุการณ์ของงานฉุกเฉิน โดยใช้ **Stack** ในการเก็บประวัติการทำงาน และใช้ **State** เพื่อบอกสถานะปัจจุบันของระบบ

1. เริ่มต้นโปรแกรม

เมื่อเปิดโปรแกรม ระบบจะเริ่มต้นที่สถานะ

NEW

และ Event Stack กับ Redo Stack จะยังว่างอยู่

2. เลือก Algorithm

ผู้ใช้สามารถเลือก Algorithm ที่ต้องการใช้งานได้ 2 แบบ

- Algorithm A: Event Stack
- Algorithm B: Event Stack + State Machine

ทั้งสอง Algorithm ใช้จัดการ Workflow เดียวกัน แต่มีวิธีการจัดการ State ต่างกัน

3. เพิ่ม Action

ผู้ใช้เลือก Action ที่ต้องการ เช่น

CALL_RECEIVED

โปรแกรมจะตรวจสอบว่า Action นี้สามารถทำได้จาก State ปัจจุบันหรือไม่ โดยตรวจสอบจาก Transition Table

ถ้าถูกต้อง โปรแกรมจะสร้าง Event และนำไปเก็บใน Event Stack

ตัวอย่าง:

NEW → CALL_RECEIVED → RECEIVED

4. ทำ Action ต่อไป

เมื่อ Action แรกสำเร็จ ผู้ใช้สามารถทำ Action ต่อไปตามลำดับ เช่น

TEAM_ASSIGNED

ระบบจะตรวจสอบอีกครั้งว่า

RECEIVED → TEAM_ASSIGNED

สามารถทำได้หรือไม่

ถ้าถูกต้อง State จะเปลี่ยนเป็น

ASSIGNED

5. ทำงานตามลำดับจนจบ

ผู้ใช้สามารถทำ Action ต่อไปตามลำดับดังนี้

CALL_RECEIVED



TEAM_ASSIGNED



VEHICLE_DISPATCHED



ARRIVED_AT_SCENE



CASE_CLOSED

เมื่อทำครบ State สุดท้ายจะเป็น

CLOSED

6. กรณีเลือก Action ผิดลำดับ

ถ้าผู้ใช้เลือก Action ที่ไม่สามารถทำได้จาก State ปัจจุบัน โปรแกรมจะไม่เพิ่ม Action นั้นลง Stack ตัวอย่างเช่น เริ่มต้นที่

NEW

แต่เลือก

TEAM_ASSIGNED

โปรแกรมจะแจ้งว่า Action ไม่ถูกต้อง และ State ยังคงเป็น

NEW

7. การใช้ Undo

เมื่อผู้ใช้กด Undo โปรแกรมจะนำ Event ล่าสุดออกจาก Event Stack แล้วนำไปเก็บไว้ใน Redo Stack ตัวอย่าง:

NEW → RECEIVED → ASSIGNED

ถ้า Undo 1 ครั้ง

NEW → RECEIVED

แสดงว่า Action ล่าสุดถูกย้อนกลับแล้ว

8. การใช้ Redo

เมื่อผู้ใช้กด **Redo** โปรแกรมจะนำ Event ที่อยู่ใน Redo Stack กลับมาทำงานอีกครั้ง

ตัวอย่าง:

ก่อน Redo:

NEW → RECEIVED

เมื่อ Redo:

NEW → RECEIVED → ASSIGNED

ดังนั้น State จะกลับมาเป็น ASSIGNED

9. กรณี Undo แล้วเพิ่ม Action ใหม่

ถ้าผู้ใช้ Undo แล้วเลือก Action ใหม่ โปรแกรมจะล้าง Redo Stack

ตัวอย่าง:

NEW → RECEIVED → ASSIGNED

จากนั้น Undo:

NEW → RECEIVED

ถ้าผู้ใช้เพิ่ม Action ใหม่ ระบบจะสร้างเส้นทางการทำงานใหม่ และไม่สามารถ Redo Action เดิมได้

10. ดูข้อมูลใน Stack

ผู้ใช้สามารถเลือกเมนู Stack เพื่อดู

- Current State
- Event Stack
- Redo Stack

ทำให้สามารถตรวจสอบได้ว่าระบบมีประวัติการทำงานอะไรอยู่บ้าง

11. Reset

เมื่อผู้ใช้เลือก **Reset** ระบบจะล้างข้อมูลทั้งหมดและกลับไปเริ่มต้นที่

NEW

พร้อมกับล้าง Event Stack และ Redo Stack

12. การทดสอบโปรแกรม

โปรแกรมมี Test Cases สำหรับตรวจสอบการทำงาน เช่น

- Workflow ปกติ
- Action ผิดลำดับ
- Undo
- Undo หลายครั้ง
- Redo หลายครั้ง
- Stack ว่าง
- เพิ่ม Action ใหม่หลัง Undo
- การเปรียบเทียบ Algorithm A และ B

เพื่อให้แน่ใจว่าโปรแกรมทำงานถูกต้องตามที่ออกแบบไว้

สรุปการทำงาน

โดยรวมโปรแกรมทำงานตามลำดับดังนี้

เริ่มต้น → เลือก Algorithm → เพิ่ม Action → ตรวจสอบ State → บันทึก Event ลง Stack →
เปลี่ยน State → สามารถ Undo/Redo → ตรวจสอบผลลัพธ์

ระบบจะไม่อนุญาตให้ผู้ใช้งานทำ Action ผิดลำดับ และสามารถย้อนกลับหรือทำ Action ที่ถูก Undo กลับมาใหม่ได้

บทที่ 4 การอธิบายความถูกต้อง

4.1 Precondition

ก่อนเริ่มทำงาน Stack และ Redo Stack ต้องพร้อมใช้งาน State เริ่มต้นเป็น NEW Action
ที่รับเข้าต้องเป็น Action ที่ระบบรู้จัก และ Transition Table ต้องถูกสร้างก่อนดำเนินการ

4.2 Invariant

1. Event ทุกตัวใน Stack ต้องเป็น Event ที่เกิดตามลำดับ Workflow ที่ถูกต้อง
2. event.previousState คือ State ก่อนเกิด Action และ event.nextState คือ State หลังเกิด Action
3. State ปัจจุบันต้องสอดคล้องกับ Event ล่าสุดใน Stack สำหรับ Algorithm A ใช้ top.nextState และถ้า

Stack ว่างเป็น NEW ส่วน Algorithm B ใช้ stored currentState

4. Redo Stack ต้องเก็บ Event ที่ถูก Undo ตามลำดับ LIFO

4.3 Postcondition

หลัง addAction สำเร็จ Event ใหม่ถูกเพิ่มลง Stack State เปลี่ยนไปยัง State ถัดไป และ Redo Stack ถูกล้าง

หลัง undo สำเร็จ Event ล่าสุดถูกย้ายจาก Stack ไป Redo Stack และ State ย้อนกลับ

หลัง redo สำเร็จ Event ถูกย้ายกลับจาก Redo Stack ไป Stack และ State เปลี่ยนกลับไปยัง Next State

4.4 Termination

โปรแกรมสิ้นสุดเมื่อผู้ใช้เลือก Exit ส่วน Add, Undo และ Redo จะสิ้นสุดหลังตรวจสอบเงื่อนไขและดำเนินการกับข้อมูลตำแหน่งบนสุดของ Stack เสร็จสิ้น

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

5.1 Best Case

Push Event, Pop Event, Peek และการตรวจสอบ Transition สามารถพิจารณาได้เป็น $O(1)$

5.2 Average Case

การเพิ่ม Action ประกอบด้วยการตรวจสอบ Action/Transition การสร้าง Event การ Push และการล้าง Redo Stack โดยการดำเนินการหลักของ Stack และ Transition ที่มีขนาดจำกัดสามารถพิจารณาเป็น $O(1)$

5.3 Worst Case

Push, Pop และ Peek เป็น $O(1)$ แต่การ clearRedo() อาจมีค่าใช้จ่ายตามจำนวนข้อมูลที่อยู่ใน Redo Stack

5.4 Time Complexity

5.5 Space Complexity

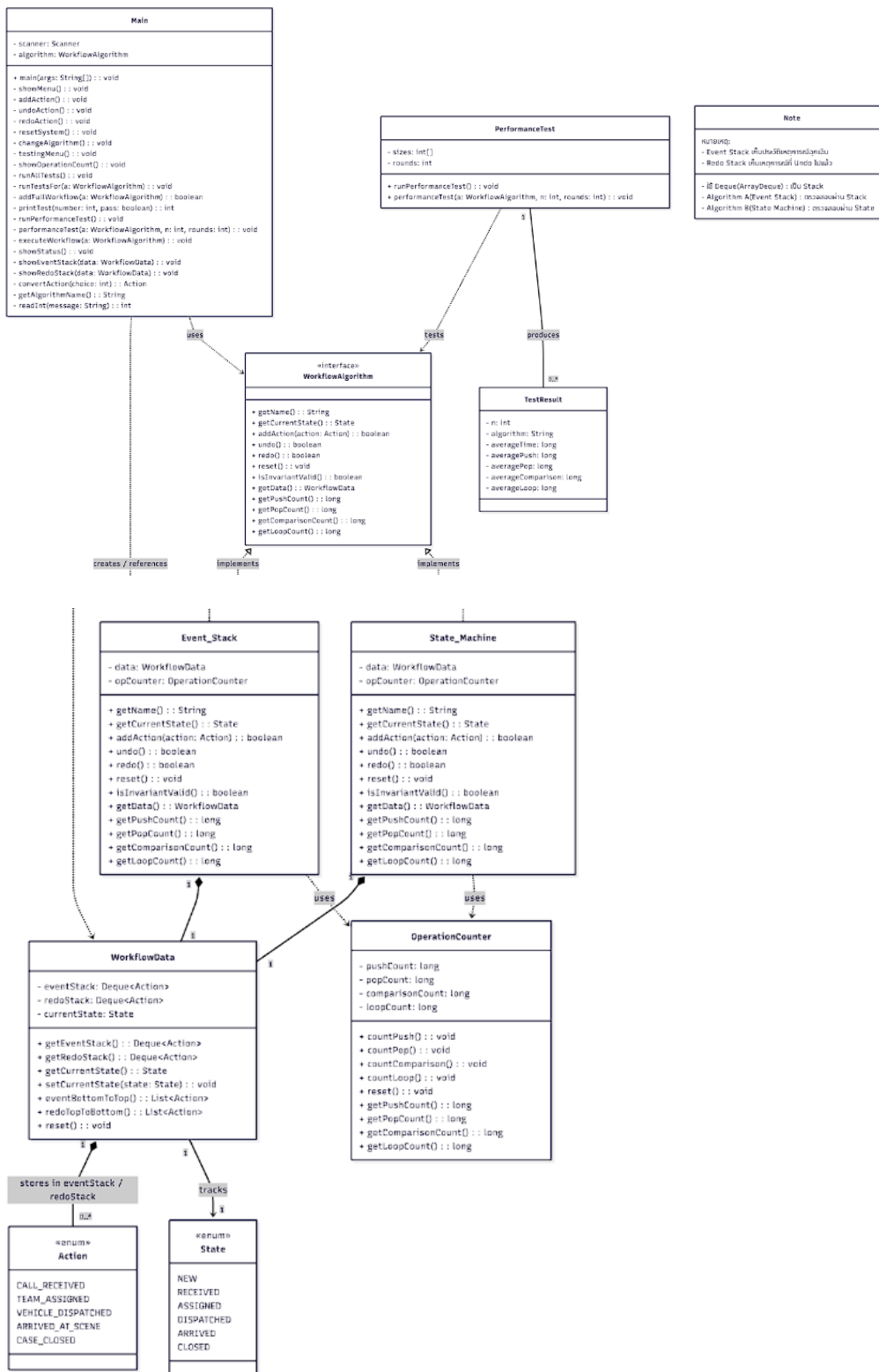
ให้ n เป็นจำนวน Event ที่จัดเก็บ ระบบใช้ Event Stack และ Redo Stack จึงมี Space Complexity โดยรวม $O(n)$ สำหรับ Algorithm B มี currentState เพิ่ม แต่ใช้พื้นที่คงที่และไม่เปลี่ยน Big-O

5.6 ตารางเปรียบเทียบ

Operation	Algorithm A	Algorithm B
Add Action	$O(1)$	$O(1)$
Undo	$O(1)$	$O(1)$
Redo	$O(1)$	$O(1)$
Peek State	$O(1)$	$O(1)$
Reset	$O(n)$	$O(n)$
Stack Push	$O(1)$	$O(1)$
Stack Pop	$O(1)$	$O(1)$

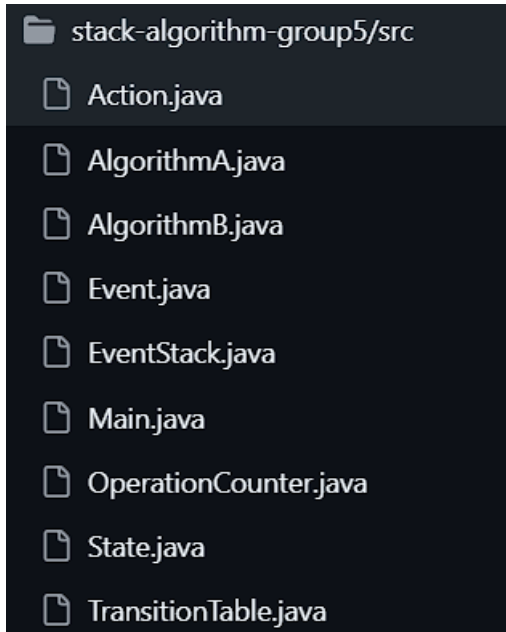
คุณสมบัติ	Algorithm A	Algorithm B
ใช้ Event Stack	✓	✓
ใช้ Redo Stack	✓	✓
ใช้ State Machine	✓	✓
เก็บ Current State โดยตรง	✗	✓
Undo / Redo	✓ / ✓	✓ / ✓
Time Complexity หลัก	$O(1)$	$O(1)$
Space Complexity	$O(n)$	$O(n)$
การเข้าถึง State	จาก Stack	จากตัวแปร currentState

บทที่ 6 การพัฒนาโปรแกรม Java



6.1 Class Diagram

โปรแกรมประกอบด้วยส่วนที่จัดการ Algorithm, Event, State, Stack และ Transition โดย Algorithm A และ B แตกต่างกันในวิธีการจัดการ State ปัจจุบัน



6.2 โครงสร้างคลาส

รูปที่ 6.3: เมธอด addAction()

```
public boolean addAction(Action action) {  
    if (action == null) return false;  
  
    State next = table.nextState(currentState, action);  
    counter.comparison();  
  
    if (next == null) return false;  
  
    stack.push(new Event(action, currentState, next));  
    counter.push();  
  
    currentState = next;  
    stack.clearRedo();  
  
    return true;  
}
```

รูปที่ 6.4: เมธอด undo()

```
public boolean undo() {
    Event event = stack.pop();

    if (event == null) return false;

    counter.pop();

    stack.pushRedo(event);
    counter.push();

    currentState = event.getPreviousState();

    return true;
}
```

รูปที่ 6.5: เมธอด redo()

```
public boolean redo() {
    Event event = stack.popRedo();

    if (event == null) return false;

    counter.pop();

    State next = table.nextState(currentState, event.getAction());
    counter.comparison();

    if (next != event.getNextState()) {
        stack.pushRedo(event);
        counter.push();
        return false;
    }

    stack.push(event);
    counter.push();

    currentState = next;

    return true;
}
```


6.3 เมฆอดคำคัญ

เมฆอดหลัก ได้แก่ addAction(), undo(), redo(), getCurrentState() และ reset()

โดยทำหน้าทึตามลำดับในการเพิ่ม Event, ย้อนกลับ, ทำซ้ำ, ตรวจสอบสถานะ และคึนระบบสู่สถานะเริ่มต้น

```
Reset แล้ว State = NEW

----- MENU -----
Algorithm : B - Event Stack + State Machine
State      : NEW
1. เลือก Algorithm
2. เพิ่ม Action
3. Undo
4. Redo
5. ดู Stack / State
6. Reset
7. Test Cases
8. เปรียบเทียบ A กับ B
9. Performance Test
0. Exit
เลือก: 2

Action:
1. CALL_RECEIVED
2. TEAM_ASSIGNED
3. VEHICLE_DISPATCHED
4. ARRIVED_AT_SCENE
5. CASE_CLOSED
เลือก Action: █

Action:
1. CALL_RECEIVED
2. TEAM_ASSIGNED
3. VEHICLE_DISPATCHED
4. ARRIVED_AT_SCENE
5. CASE_CLOSED
เลือก Action: 4
เพิ่มไม่สำเร็จ: Action ผิดลำดับ
State ปัจจุบัน: NEW

----- MENU -----
Algorithm : B - Event Stack + State Machine
State      : NEW
1. เลือก Algorithm
2. เพิ่ม Action
3. Undo
4. Redo
5. ดู Stack / State
6. Reset
7. Test Cases
8. เปรียบเทียบ A กับ B
9. Performance Test
0. Exit
เลือก: █
```

6.4 Input Validation

ตรวจสอบว่า Action เป็นค่าที่ระบบรู้จักและสามารถเปลี่ยนจาก State ปัจจุบันไปยัง State ถัดไปตาม Transition Table ได้ หากไม่ถูกต้องจะปฏิเสธและไม่เพิ่ม Event ลง Stack

```
เลือก: 3

Undo ไม่ได้: Stack ว่าง
State ปัจจุบัน: NEW

----- MENU -----
Algorithm : B - Event Stack + State Machine
State      : NEW
1. เลือก Algorithm
2. เพิ่ม Action
3. Undo
4. Redo
5. ดู Stack / State
6. Reset
7. Test Cases
8. เปรียบเทียบ A กับ B
9. Performance Test
0. Exit

เลือก: 4

Redo ไม่ได้: Redo Stack ว่างหรือ State ไม่ถูกต้อง
State ปัจจุบัน: NEW

----- MENU -----
Algorithm : B - Event Stack + State Machine
State      : NEW
1. เลือก Algorithm
2. เพิ่ม Action
3. Undo
4. Redo
5. ดู Stack / State
6. Reset
7. Test Cases
8. เปรียบเทียบ A กับ B
9. Performance Test
0. Exit
เลือก: █
```

6.5 Error Handling

รองรับกรณี Undo เมื่อ Stack ว่าง, Redo เมื่อ Redo Stack ว่าง และ Action ไม่ถูกต้องตาม Workflow โดยไม่เปลี่ยนแปลง State เมื่อการดำเนินการไม่สำเร็จ

องค์ประกอบ	หน้าที่
Algorithm A	จัดการ Event โดยใช้ Stack เป็นแหล่งข้อมูลหลักในการหาสถานะปัจจุบัน
Algorithm B	จัดการ Event โดยใช้ Stack ร่วมกับตัวแปร currentState
Stack	จัดเก็บ Event ตามลำดับการทำงาน
Redo Stack	จัดเก็บ Event ที่ถูก Undo เพื่อรองรับ Redo
Event	เก็บ Action, Previous State และ Next State

Transition Table	กำหนด State ถัดไปจาก State และ Action
------------------	---------------------------------------

บทที่ 7 การทดลอง

7.1 ขนาดข้อมูล

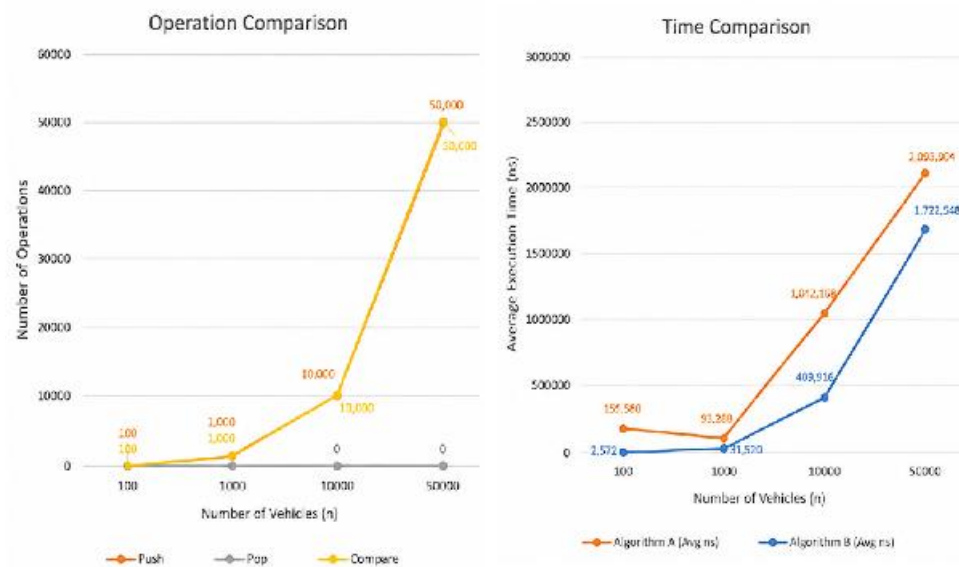
ใช้ขนาดข้อมูล 100, 1,000, 10,000 และ 50,000 Event เพื่อเปรียบเทียบ Algorithm A และ Algorithm B

7.2 จำนวนรอบทดลอง

ทดลองแต่ละขนาดข้อมูล 5 รอบ และใช้ System.nanoTime() วัดเวลา

7.3 ตารางผลลัพธ์

บันทึกค่าที่วัดได้ของ Algorithm A และ Algorithm B ในแต่ละขนาดข้อมูล และใช้ค่าเฉลี่ยจากผลการทดลองจริง



7.4 กราฟ

นำผลการทดลองจริงมาแสดงเป็นกราฟเพื่อดูแนวโน้มและความแตกต่างของประสิทธิภาพ

7.5 การอภิปรายผล

ทั้งสองอัลกอริทึมใช้ Stack และ Redo Stack เหมือนกัน ความแตกต่างหลักคือ Algorithm A อาศัย Stack เพื่ออ่าน State ส่วน Algorithm B เก็บ currentState โดยตรง ทั้งสองมี Time Complexity หลัก $O(1)$ และ Space Complexity $O(n)$ ผลเวลาจริงควรพิจารณาพร้อมกับค่าเฉลี่ยจาก 5 รอบและสภาพแวดล้อมของเครื่อง

ครั้ง	จำนวนข้อมูล
1	100
2	1,000
3	10,000
4	50,000

ขนาดข้อมูล	Algorithm A รอบ 1-5	A Average (ns)	Algorithm B รอบ 1-5	B Average (ns)
100	กรอกผลจริง	136672	กรอกผลจริง	5684
1,000	กรอกผลจริง	93288	กรอกผลจริง	31484
10,000	กรอกผลจริง	661168	กรอกผลจริง	409916
50,000	กรอกผลจริง	2093904	กรอกผลจริง	1720948

บทที่ 8 สรุป

8.1 วิธีที่เหมาะสมกว่า

Algorithm A เหมาะสำหรับแสดงแนวคิด Stack เป็นแหล่งข้อมูลหลัก ส่วน Algorithm B มีข้อได้เปรียบในการเข้าถึง State ปัจจุบันโดยตรง การเลือกใช้จริงควรพิจารณาพร้อมกับผลการทดลองของกลุ่ม

8.2 เหตุผลในการเลือก

พิจารณาจากความเข้าใจง่าย ความสอดคล้องกับแนวคิด Stack การจัดการ Current State การรองรับ Undo/Redo และผลการทดลองด้านประสิทธิภาพ

8.3 ข้อจำกัด

ระบบทำงานภายใต้ State และ Transition ที่กำหนดไว้
และผลการวัดประสิทธิภาพอาจขึ้นกับสภาพแวดล้อมของเครื่อง จำนวนข้อมูล และวิธีการทดลอง

8.4 แนวทางพัฒนาต่อ

สามารถเพิ่ม Workflow และ State ที่หลากหลาย เพิ่มส่วนติดต่อผู้ใช้ ปรับปรุงการจัดเก็บ Event และเพิ่มรูปแบบการทดลอง

8.5 สิ่งที่ได้เรียนรู้

ได้เรียนรู้การประยุกต์ใช้ Stack กับการจัดการ Event และ Undo/Redo รวมถึงการออกแบบ Algorithm, Pseudocode, การอธิบายความถูกต้อง การวิเคราะห์ Time/Space Complexity และการทดลองเปรียบเทียบประสิทธิภาพ

หมายเหตุ: เอกสารฉบับนี้เรียบเรียงจากข้อมูลในรายงาน Word ฉบับล่าสุดที่ผู้ใช้ส่งมา

โดยคงเฉพาะข้อมูลที่มีอยู่ และ ไม่เติมผลการทดลองหรือรายละเอียดจากแหล่งอื่นที่ยังไม่มีในเอกสาร

คู่มือการใส่รูปจาก GitHub

- บทที่ 3: ใช้รูป Flowchart และ Screenshot การทำงานจริงของ Stack ได้แก่ เริ่มต้น, Add/Push, Undo และ Redo
- บทที่ 6: ใช้ Screenshot โครงสร้างไฟล์จาก GitHub, Class Diagram และ Screenshot เมฆอดสำคัญ
- บทที่ 6.4–6.5: ใช้ Screenshot ผลการรันจริงสำหรับ Input Validation และ Error Handling
- บทที่ 7.4: ใช้กราฟจากผลการทดลองจริง ไม่ควรสร้างตัวเลขขึ้นเอง